

Entrega #3 Mini-lenguaje

Jesús Ismael Ramos Recio

Cubo Semántico

El siguiente fragmento de código muestra la implementación del cubo semántico que define las reglas de tipos para las operaciones del lenguaje y cuál sería el tipo resultante de una operación según los tipos de los operandos. Se utilizan las matrices para representar con claridad el tipo resultante de una operación con 2 factores que son los tipos del operando izquierdo y el operando derecho.

```
class CuboSemantico:
    # Definicion de reglas de tipos para operaciones

    def __init__(self):
        # Operadores aritmeticos (+, -, *)
        self.aritmeticos = {
            ('int', 'int'): 'int',
            ('int', 'float'): 'float',
            ('float', 'int'): 'float',
            ('float', 'float'): 'float',
        }

        # Division (/)
        # Siempre retorna flotantes
        self.division = {
            ('int', 'int'): 'float',
            ('int', 'float'): 'float',
            ('float', 'int'): 'float',
            ('float', 'float'): 'float',
        }

        # Operadores relacionales (==, !=, <, >, <=, >=)
        # Booleano numerico (1 = verdadero, 0 = falso)
        self.relacionales = {
            ('int', 'int'): 'int',
            ('int', 'float'): 'int',
            ('float', 'int'): 'int',
            ('float', 'float'): 'int',
        }

        # Asignacion (=)
        self.asignacion = {
            ('int', 'int'): True,
            ('int', 'float'): True, # Truncamiento
            ('float', 'int'): True,
            ('float', 'float'): True,
        }

    def tipo_aritmetico(self, tip_izq, tip_der, operador):
        # Determinar el tipo resultante de una operacion
```

```

        if operador == '/':
            return self.division.get((tip_izq, tip_der))
        else:
            return self.aritmeticos.get((tip_izq, tip_der))

    def tipo_relacional(self, tip_izq, tip_der, operador):
        # Determinar el tipo resultante de una comparacion
        # Primero validar que los tipos sean compatibles
        if (tip_izq, tip_der) in self.relacionales:
            return 'int'
        return None

    def asignacion_valida(self, tip_var, tip_exp):
        # Verificar si la asignacion es valida
        return self.asignacion.get((tip_var, tip_exp), False)

    def promocion(self, tip_izq, tip_der):
        # Determinar el tipo comun para promocion de operandos.
        # 'float' si alguno es flotante. Si no, 'int'
        if tip_izq == 'float' or tip_der == 'float':
            return 'float'
        return 'int'

```

Puntos importantes:

- La división tiene su propia matriz debido a que en cualquier operación con operandos de cualquier tipo, siempre se generará un valor flotante.
- La función “tipo_aritmético” se ocupa de determinar el tipo del valor resultante al operarse cualquier operación aritmética.
- La función “tipo_relacional” se ocupa de determinar el tipo del valor resultante al hacer una comparación entre 2 operandos. Estos producen un resultado lógico, y en el lenguaje se diseñó que el resultado se presente como booleano numérico, es decir, en enteros donde 1 es igual a Verdadero y 0 es igual a Falso.
- La función “asignacion_valida” verifica si una asignación es válida entre 2 operandos. Se debe definir lo que se produce al asignar 2 operandos, ya que el resultado varía dependiendo de los tipos de los operandos. Por ejemplo, si se hace una asignación y el operando izquierdo es entero y el operando derecho es flotante, el resultado se volverá entero pero los decimales se perderán por medio de un truncamiento.
- La función “promoción” determina el tipo que se convertirían los operandos antes de realizar cualquier operación.

Pilas y Fila de Cuádruplos

Se implementaron 4 pilas. Una de ellas es una clase base de pilas que maneja las funciones de cualquier pila. La implementación de esta sirve para reducir trabajo de código para las 3 pilas especializadas.

Las pilas especializadas se describen a continuación:

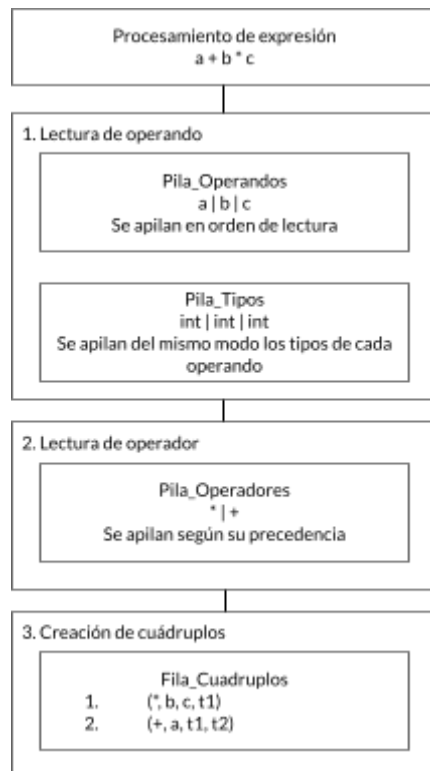
- **Pila_Operadores:** Almacena los operadores durante el análisis de expresiones. Tiene la función “mayor_precedencia” para comparar la jerarquía entre 2 operadores, la cual fue definida en el diccionario “precedencia”. La forma en la que funciona la prioridad de operadores, es que se les asigna a cada operador un número. El operador con el número más alto es el que tiene mayor prioridad.
“UMENOS” (menos unario) tiene mayor prioridad ya que se le aplica a un solo operando. Entonces cuando se use para hacer una operación, se debe aplicar antes para obtener un resultado correcto. Al generar los cuádruplos, si se encuentra un operando con un menos unario, primero se debe evaluar dicho operando
Los paréntesis funcionan como delimitadores que cambian la estructura de la expresión. Al momento de que se encuentra un ‘(’, se apila inmediatamente y se realizan los cuádruplos dentro de los paréntesis. Ya cuando se encuentra un ‘)’, se desapila todo y seguimos con la generación de los cuádruplos pendientes.
- **Pila_Operandos:** Almacena los variables, constantes y temporales. Contiene las funciones “nuevo_temporal” que genera los identificadores únicos (t1, t2, ..., tn), y “push_temp” que crea un temporal y lo agrega a la pila.
- **Pila_Tipos:** Almacena el tipo de cada operando. Contiene la función “compatible” para verificar la compatibilidad de tipos

La **Fila de Cuádruplos** solamente se encarga de agregar un cuádruplo a la fila y lo retorna. Debe seguir el método FIFO (First In, First Out) ya que los cuádruplos deben ejecutarse en el mismo orden que se generan.

Los cuádruplos siguen el siguiente formato: ‘operador’, operando izq, operando der, resultado)

Puntos Neurálgicos de las Pilas

El siguiente diagrama demuestra el proceso que siguen las pilas para generar los cuádruplos.



Traducción a cuádruplos

El algoritmo implementado para la traducción a cuádruplos utiliza las pilas y procesa las expresiones respetando la precedencia de operadores. La secuencia que se sigue para llevar a cabo la traducción es:

1. Se apila un operando y su tipo en sus respectivas pilas.
2. Después se lee un operador. Este proceso empieza comparando ese operador con el tope de la pila de operadores. Si el nuevo operador es de menor o igual prioridad, se saca de la pila y genera un cuádruplo
3. Cuando se lee un paréntesis de apertura, este se apila como marcador, y se leen los siguientes operadores dentro del marcador.
4. El momento en que se lee un paréntesis de cierre, se desapila y generan los cuádruplos dentro de los marcadores.
5. Al finalizar, los operadores pendientes se desapilan y se genera el resultado final.

Puntos Neurálgicos

El siguiente diagrama demuestra los puntos neurálgicos del algoritmo de traducción.



Estatutos lineales

Los estatutos como la asignación, la impresión y el retorno siguen el siguiente proceso para su traducción:

Asignación:

- Se verifica la compatibilidad de los tipos con el cubo semántico
- Si de un valor flotante cambia a entero, se genera el cuádruplo de 'trunc' (truncamiento)
- Se genera el cuádruplo de asignación (=, resultado de expresión, None, tn)

Impresión

- Generar el cuádruplo 'print' ('print', resultado de expresión, None, None)

Retorno

- Generar el cuádruplo 'return' ('return', resultado de expresión, None, None)

Despliegue de Cuádruplos

Esta sección mostrará el despliegue de la generación de cuádruplos. Para ello se llevó a cabo la siguiente prueba en el archivo 'prueba_cuad.ml'. Donde se lee su contenido y se analizan las expresiones y operaciones para llevar a cabo la generación de los cuádruplos.

Prueba 1

program test:

```
var int x;  
var float y;  
var int z;
```

start

```
x = 5;  
y = 3.14;  
z = y;  
print(z);
```

end

Salida:

Iniciando: prueba_cuad.ml

Información sobre variables, directorio de funciones y tabla de variables se despliega también. Se omite esta información para dar mayor importancia al despliegue de la generación de cuádruplos

CUADRUPLOS

0: (=, 5, None, x)

1: (=, 3.14, None, y)

2: (trunc, y, None, t1)

3: (=, t1, None, z)

4: (print, z, None, None)